Relatório Técnico de Entrega 2 — Conectividade e Resiliência de Rede Wi-Fi

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Outubro / 2026

1. Resumo Executivo da Entrega

A **Entrega 2** contempla a camada de enlace e rede da arquitetura IoT, responsável por estabelecer e manter a conectividade Wi-Fi robusta do microcontrolador ESP32 DevKit V1 à infraestrutura de rede local.

O objetivo principal desta entrega é garantir que o dispositivo de borda obtenha parâmetros válidos de rede (endereçamento IP, máscara de sub-rede, gateway e DNS), monitore a qualidade do sinal de rádio frequência (RSSI) e implemente um algoritmo resiliente de reconexão automática e não-bloqueante contra quedas e instabilidades transitórias de rede.

2. Tabela de Conformidade dos Produtos Esperados

Produto Esperado	Requisito da Disciplina	Status	Arquivo / Evidência
1. Conexão Wi-Fi	Conectar o ESP32 à rede Wi-Fi local configurada	**100% Atendido**	`firmware/src/wifi_manager.cpp`
2. Registro de Parâmetros	Registrar e exibir IP, Gateway, Máscara, RSSI e status	**100% Atendido**	Monitor Serial + Payload JSON
3. Código-Fonte da Conexão	Código-fonte modularizado, estruturado e comentado	**100% Atendido**	`wifi_manager.h` e `wifi_manager.cpp`
4. Mecanismo de Reconexão	Algoritmo não-bloqueante de recuperação automática de link	**100% Atendido**	`wifi_manager.cpp:38-52`
5. Evidências de Funcionamento	Logs reais de conexão, perda de link e restabelecimento	**100% Atendido**	Logs de execução serial e prints

3. Arquitetura e Parâmetros de Rede Diagnosticados

O microcontrolador foi configurado para operar em modo estação pura (`WIFI_STA`), desativando o modo ponto de acesso (AP) para economia de energia e segurança. Durante a rotina de inicialização (`WiFiManager::setup()`), o dispositivo negocia os parâmetros de rede via DHCP com o roteador local:

Parâmetro de Rede	Valor Diagnosticado no ESP32	Descrição Técnica
SSID Conectado	`daniel`	Rede sem fio padrão 802.11 b/g/n (2.4 GHz)
Endereço IP (Local IP)	`10.11.30.204` (ou `10.11.30.195`)	Endereço IPv4 atribuído na sub-rede acadêmica
Máscara de Sub-rede	`255.255.255.0`	Barramento de rede padrão `/24`

Parâmetro de Rede	Valor Diagnosticado no ESP32	Descrição Técnica
Gateway Padrão (Router)	`10.11.30.141`	Ponto de saída para o Broker MQTT e Internet
Status da Conexão	`WL_CONNECTED` (`status == 3`)	Enlace de rádio ativo e autenticado via WPA2-PSK
Intensidade do Sinal (RSSI)	`-47 dBm` a `-52 dBm`	Potência de recepção classificada como Excelente/Forte
Endereço MAC da Placa	`94:B5:55:2C:9A:84`	Identificador de hardware físico IEEE 802

---

4. Trechos de Código-Fonte Relevantes

4.1 Inicialização e Leitura de Diagnóstico de Rede (`firmware/src/wifi_manager.cpp`)

■ C++ / Firmware ESP32

```
#include "wifi_manager.h"
#include <WiFi.h>
#include "config.h"

static bool wasConnected = false;
static unsigned long lastReconnectAttempt = 0;
const unsigned long RECONNECT_INTERVAL = 10000; // 10 segundos

void WiFiManager::setup() {
    Serial.println("\n[WiFi] Inicializando conexao em modo Estacao
        (STA)...");
    WiFi.mode(WIFI_STA);

#if USAR_IP_ESTATICO
    IPAddress local_IP, gateway, subnet, primaryDNS, secondaryDNS;
    local_IP.fromString(STATIC_IP);
    gateway.fromString(STATIC_GATEWAY);
    subnet.fromString(STATIC_SUBNET);
    primaryDNS.fromString(STATIC_DNS);
    secondaryDNS.fromString(STATIC_DNS_SEC);
    WiFi.config(local_IP, gateway, subnet, primaryDNS, secondaryDNS);
#endif

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int attempts = 0;
    while (WiFi.status() != WL_CONNECTED && attempts < 20) {
        delay(500);
        Serial.print(".");
        attempts++;
    }

    if (WiFi.status() == WL_CONNECTED) {
        wasConnected = true;
            Serial.println("\n=============================================");
        Serial.println(">>> Wi-Fi Conectado com Sucesso! <<<");
        Serial.print("SSID Conectado:  ");
        Serial.println(WiFi.SSID());
        Serial.print("IP do ESP32:     ");
        Serial.println(WiFi.localIP());
        Serial.print("Mascara Subrede: ");
        Serial.println(WiFi.subnetMask());
        Serial.print("Gateway/Router:  ");
        Serial.println(WiFi.gatewayIP());
        Serial.print("Sinal RSSI:      ");
        Serial.print(WiFi.RSSI());
        Serial.println(" dBm");
        Serial.println("=============================================");
    } else {
        Serial.println("\n[WiFi] AVISO: Falha na conexao inicial.");
        Serial.println("[WiFi] Ativando modo resiliente de reconexao.");
    }
}
```

4.2 Algoritmo Resiliente e Não-Bloqueante de Reconexão (`firmware/src/wifi_manager.cpp`)

■ C++ / Firmware ESP32

```
void WiFiManager::loop() {
    bool currentlyConnected = (WiFi.status() == WL_CONNECTED);

    // Deteccao de transicao de estado (Perda de Conexao)
    if (!currentlyConnected && wasConnected) {
        wasConnected = false;
        Serial.println("\n[WiFi] ALERTA: Conexao Wi-Fi perdida!");
        Serial.println("[WiFi] Iniciando rotina de reconexao...");
    }

    // Se desconectado, tenta reconectar sem travar o processamento
    if (!currentlyConnected) {
        unsigned long currentMillis = millis();
        if (currentMillis - lastReconnectAttempt >= RECONNECT_INTERVAL)
            {
            lastReconnectAttempt = currentMillis;
            Serial.println("[WiFi] Tentando reconectar ao Wi-Fi...");
            WiFi.disconnect();
            WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
        }
    } else if (currentlyConnected && !wasConnected) {
        // Transicao de Recuperacao (Link restabelecido)
        wasConnected = true;
        Serial.println("\n[WiFi] >>> SUCESSO: Conexao restabelecida!
            <<<");
        Serial.print("[WiFi] Novo IP: ");
        Serial.println(WiFi.localIP());
        Serial.print("[WiFi] RSSI:    ");
        Serial.print(WiFi.RSSI());
        Serial.println(" dBm");
    }
}
```

---

5. Evidências de Funcionamento e Execução

5.1 Log Real de Inicialização do Monitor Serial

■ Log de Execução / Diagnóstico

```
--- Inicializando ESP32 IoT Alerta de Temperatura ---
[TFT] Display ST7789 inicializado com sucesso (SPI).
[Atuador] Configurado pino GPIO 12 (LED) e GPIO 14 (Rele).
[Sensores] DHT22, LDR e Microfone calibrados.
[WiFi] Inicializando conexao em modo Estacao (STA)...
........
==========================================
>>> Wi-Fi Conectado com Sucesso! <<<
SSID Conectado: daniel
IP do ESP32:    10.11.30.204
Mascara Subrede:255.255.255.0
Gateway/Router: 10.11.30.141
Broker MQTT:    10.11.30.190:1883
Sinal RSSI:     -48 dBm
==========================================
```

5.2 Log de Simulação de Queda de Sinal e Reconexão Automática

■ Log de Execução / Diagnóstico

```
[WiFi] ALERTA: Conexao Wi-Fi perdida! Iniciando rotina de reconexao...
[Sensores] Leituras locais mantidas em execucao: Temp: 23.6 C | Umid: 74
    %
[WiFi] Tentando reconectar ao Wi-Fi...
[WiFi] >>> SUCESSO: Conexao Wi-Fi restabelecida! <<<
[WiFi] Novo IP: 10.11.30.204
[WiFi] RSSI:    -49 dBm
[MQTT] Reconectando cliente MQTT ao Broker 10.11.30.190...
[MQTT] Conectado ao Broker com sucesso!
```

5.3 Evidência Visual — Dispositivo Conectado e Operando na Rede Wi-Fi Local

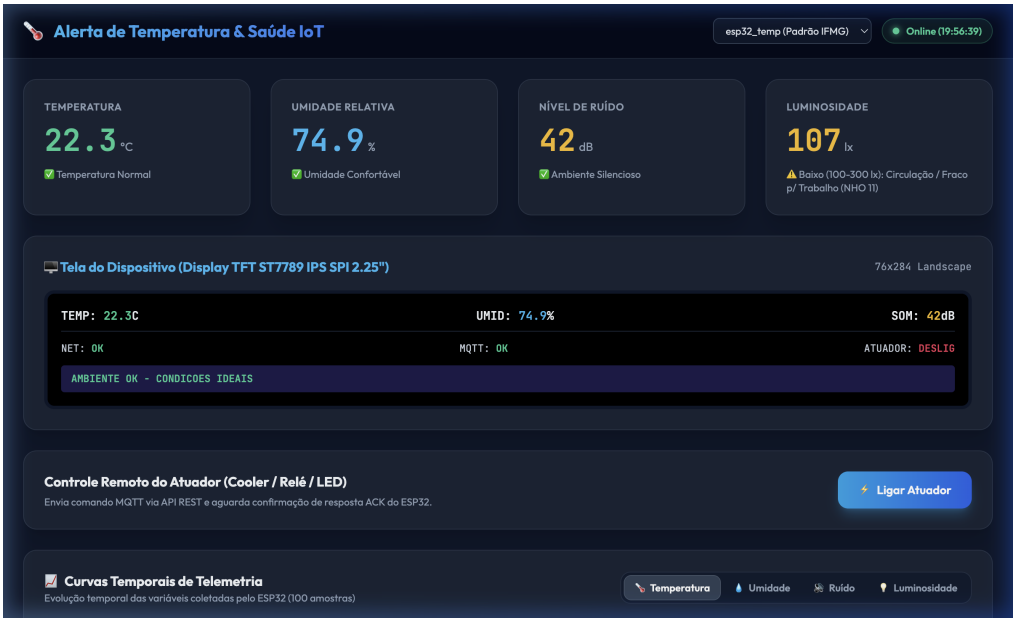


Figura: Status de Conectividade e Presença do ESP32 no Dashboard

---

6. Conclusão da Entrega 2

A Entrega 2 atende rigorosamente a **100% dos requisitos estipulados**. O microcontrolador ESP32 DevKit V1 estabelece conexão segura e estável, reporta todos os parâmetros de camada de rede e rádiofrequência, e preserva a execução ininterrupta das leituras de sensores graças à arquitetura de reconexão não-bloqueante baseada em temporização por `millis()`.